AWS システム設計資料 - Webサイト＆ブログホスティングシステム

システム概要

静的なWebコンテンツ（HTML、CSS、JavaScript、画像）をAWSで高速・スケーラブルに提供するシステム。CDN (CloudFront) を活用することで、世界中からのアクセスを低遅延で配信できます。

対象用途:

- 企業ウェブサイト
- ブログプラットフォーム
- ドキュメントサイト
- ポートフォリオサイト
- ランディングページ

アーキテクチャ構成

```
インターネット
    ↓
  Route 53 (DNS)
    ↓
  CloudFront (CDN)
    ├→ キャッシュ層（エッジロケーション）
    └→ S3 Bucket (オリジン)
         ├→ HTML/CSS/JS
         ├→ 画像ファイル
         └→ 静的コンテンツ
```

使用するAWSサービス

サービス	役割	主な責務
S3 (Simple Storage Service)	オリジンストレージ	Webコンテンツの保存、バージョン管理
CloudFront	CDN配信	グローバルキャッシング、高速配信
Route 53	DNS管理	ドメイン登録、ルーティング管理
CloudWatch	監視	トラフィック監視、ログ記録
Certificate Manager (ACM)	SSL/TLS	無料HTTPS証明書

詳細設定値

S3 バケット設定

```
バケット名: blog-content-production
リージョン: us-east-1（CloudFront統合の推奨）

バージョニング設定:
  - 有効: ○
  - 理由: 過去バージョンへの復元対応

ブロック公開アクセス:
  - すべての設定: ON（重要）
  - CloudFront OAI経由のみアクセス許可

暗号化:
  - SSE-S3: 有効
  - 暗号化キー: Amazon管理キー

オブジェクトロック:
  - 無効（必要に応じて有効化）

アクセスログ:
  - 有効: ○
  - ログ保存先: s3://access-logs-bucket/

ライフサイクルルール:
  - ルール名: "Archive Old Versions"
  - 非現在版を30日後にGlacierへ転送
  - 90日後に削除
```

CloudFront ディストリビューション設定

```
配信方式: Web

オリジン設定:
  - オリジンドメイン: blog-content-production.s3.us-east-1.amazonaws.com
  - カスタムヘッダー: なし
  - SSL プロトコル: TLS 1.2

オリジンアクセスコントロール (OAC):
  - OAC ID: [AWS自動生成]
  - 署名バージョン: sigv4
  - 効果: CloudFront経由のアクセスのみ許可

キャッシュの動作:
  - ビューアープロトコルポリシー: Redirect HTTP to HTTPS
  - キャッシュキーとオリジンリクエスト:
    * TTL デフォルト: 3600秒（1時間）
    * TTL 最大: 31536000秒（365日）
    * TTL 最小: 0秒
    * 自動圧縮: 有効（gzip, brotli）

ホストヘッダー: blog.example.com
```

```
クエリ文字列:
  - キャッシュ対象: すべてのパラメータを含める（必要に応じ）
  - デフォルト: キャッシュキーに含める

クッキー:
  - すべてのクッキーをキャッシュキーに含める

セキュリティ:
  - デフォルトルート オブジェクト: index.html
  - カスタムエラーレスポンス:
    * 404エラー → error/404.html（200応答）
    * 500エラー → error/500.html（500応答）

HTTPS設定:
  - SSL 証明書: ACM（無料）
  - SSLサポートメソッド: SNI only
  - TLS バージョン: TLSv1.2_2021

セキュリティ設定:
  - HTTP/2: 有効
  - HTTP/3 (QUIC): 有効
  - IPv6: 有効
```

Route 53 設定

```
ホストゾーン: example.com

Aレコード設定:
  - 名前: blog.example.com
  - レコードタイプ: A
  - エイリアス: ○
  - ターゲット: CloudFront ディストリビューション
  - ルーティングポリシー: シンプル

AAAA レコード設定 (IPv6):
  - 名前: blog.example.com
  - レコードタイプ: AAAA
  - エイリアス: ○
  - ターゲット: CloudFront ディストリビューション

TTL: 300秒（5分）
```

推定月額コスト

シナリオ: 月間トラフィック 500 GB、1000万リクエスト

項目	使用量	単価	月額費用
S3 ストレージ	20 GB	$0.023/GB	$0.46

項目	使用量	単価	月額費用
S3 GET リクエスト	1000万	$0.0004/1000req	$4.00
S3 PUT リクエスト	10万	$0.005/1000req	$0.50
CloudFront データ転送	500 GB	$0.085/GB*	$42.50
CloudFront リクエスト	1000万	$0.0075/10000req	$7.50
Route 53 ホストゾーン	1	$0.50/zone	$0.50
Route 53 クエリ	100万	$0.40/1000万	$0.04
合計			**$55.50**

*CloudFront の料金は地域によって異なります（$0.085〜$0.120/GB）

セキュリティ設定

S3 バケットポリシー例

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "AllowCloudFrontOACAccess",
      "Effect": "Allow",
      "Principal": {
        "Service": "cloudfront.amazonaws.com"
      },
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::blog-content-production/*",
      "Condition": {
        "StringEquals": {
          "AWS:SourceArn": "arn:aws:cloudfront::ACCOUNT-
ID:distribution/DISTRIBUTION-ID"
        }
      }
    }
  ]
}
```

CloudFront カスタムヘッダー（セキュリティ強化）

```
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
X-XSS-Protection: 1; mode=block
Strict-Transport-Security: max-age=31536000; includeSubDomains
Content-Security-Policy: default-src 'self'
```

デプロイメント手順

1. S3 バケット作成

```
aws s3api create-bucket \
  --bucket blog-content-production \
  --region us-east-1

# バージョニング有効化
aws s3api put-bucket-versioning \
  --bucket blog-content-production \
  --versioning-configuration Status=Enabled

# ブロック公開アクセス
aws s3api put-public-access-block \
  --bucket blog-content-production \
  --public-access-block-configuration \

"BlockPublicAcls=true,IgnorePublicAcls=true,BlockPublicPolicy=true,RestrictPublicB
uckets=true"
```

2. コンテンツアップロード

```
# 単一ファイル
aws s3 cp index.html s3://blog-content-production/

# ディレクトリ全体
aws s3 sync ./public s3://blog-content-production/ \
  --delete \
  --cache-control "max-age=3600" \
  --metadata-directive REPLACE

# キャッシュ無効化（画像など）
aws s3 cp image.png s3://blog-content-production/ \
  --cache-control "max-age=31536000,public"
```

3. CloudFront キャッシュ無効化

```
aws cloudfront create-invalidation \
  --distribution-id EXXXXXX \
  --paths "/*"

# 特定ファイルのみ
aws cloudfront create-invalidation \
  --distribution-id EXXXXXX \
  --paths "/index.html" "/about.html"
```

パフォーマンス最適化

1. 画像最適化

- WebP 形式対応（CloudFront 自動変換）
- 適応型ビットレート
- 遅延読み込み実装

2. キャッシュ戦略

```
HTML ファイル:
  - TTL: 0秒（常にオリジンから取得）
  - リビジョンハッシュ付きURL使用推奨

CSS/JavaScript:
  - TTL: 31536000秒（1年）
  - ファイル名にハッシュを含める
  - 変更時は新ファイル名で配置

画像:
  - TTL: 31536000秒（1年）
  - WebP + フォールバック

フォント:
  - TTL: 31536000秒（1年）
```

3. 圧縮設定

```
自動圧縮: 有効
  - gzip: HTML, CSS, JS
  - Brotli: テキスト系ファイル

最小サイズ: 860バイト以上
```

監視とロギング

CloudWatch メトリクス

```
監視対象:
- Requests: リクエスト数
- BytesDownloaded: ダウンロードバイト数
- BytesUploaded: アップロードバイト数
- 4xxErrorRate: 4xx エラー率
- 5xxErrorRate: 5xx エラー率
- CacheHitRate: キャッシュヒット率

アラーム設定:
```

```
  - 4xxError > 10% → 通知
  - 5xxError > 1% → 通知
  - CacheHitRate < 80% → 確認
```

アクセスログ

```
有効化: ○
ログ保存先: s3://access-logs-bucket/cloudfront-logs/
ログ形式: 標準形式
ログ更新間隔: 60分
```

トラブルシューティング

問題	原因	解決方法
404 エラー	ファイルが見つからない	デフォルトオブジェクトを確認、S3 内容確認
キャッシュが古い	TTL が長すぎる	キャッシュ無効化実行、TTL調整
HTTPS エラー	SSL 証明書問題	ACM 証明書確認、CloudFront 設定確認
遅延	キャッシュ不足	キャッシュヒット率確認、TTL最適化